

情绪对话模型训练与部署

情绪对话模型训练与部署

- 一、embedding_model验证与转换
通常embedding_model核心目录如下：
- 二、数据集转换
 1. 选择模型微调工具，根据微调框架选择对应数据集格式（此处以xtuner为例，使用xtuner默认格式）
- 三、模型选型
 1. 根据任务选择对应的评测数据，对预期模型客观评测
- 四、模型训练
 1. 配置训练文件
 2. 在当前目录下，输入以下命令启动微调脚本
 - 3.模型转换
 - 4.模型合并
- 五、模型推理部署
选择合适的大模型推理框架部署模型（这里选择LMDeploy），须注意对话模板对其操作！
- 六、模型推理部署
使用下面命令启动streamlit web前端，测试最终效果：

一、embedding_model验证与转换

通常embedding_model核心目录如下：

```
text2vec-base-chinese-sentence/  
├─ 0_Transformer/  
├─ 1_Pooling/  
├─ 2_Normalize/  
├─ config.json  
├─ config_sentence_transformers.json  
├─ modules.json  
└─ pytorch_model.bin
```

embedding_model通常情况是自带归一化层的。一般可以通过Normalize目录或者modules.json来判断是否准确包含归一化层。如果未包含归一化层，则需要手动转换添加：

```
import numpy as np  
from sentence_transformers import SentenceTransformer, models  
  
model_path = r"D:\PycharmProjects\demo_15\embedding_model\sungw111\text2vec-  
base-chinese-sentence"  
bert = models.Transformer(model_path)  
pooling = models.Pooling(bert.get_word_embedding_dimension(),  
                          pooling_mode='mean')  
  
# 添加缺失的归一化层  
normalize = models.Normalize()
```

```

# 组合完整模型
full_model = SentenceTransformer(modules=[bert, pooling, normalize])
print(full_model)

save_path=r"D:\PycharmProjects\demo_15\embedding_model\zy\text2vec-base-chinese-sentence"
full_model.save(save_path)

# 加载修复后的模型
model =
SentenceTransformer(r"D:\PycharmProjects\demo_15\embedding_model\zy\text2vec-
base-chinese-sentence")

# 验证向量归一化
text = "测试文本"
vec = model.encode(text)
print("修正后模长:", np.linalg.norm(vec)) # 应输出≈1.0

```

转换后目录如下:

转换后的modules.json如下

```

[
  {
    "idx": 0,
    "name": "0",
    "path": "",
    "type": "sentence_transformers.models.Transformer"
  },
  {
    "idx": 1,
    "name": "1",
    "path": "1_Pooling",
    "type": "sentence_transformers.models.Pooling"
  },
  {
    "idx": 2,
    "name": "2",
    "path": "2_Normalize",
    "type": "sentence_transformers.models.Normalize"
  }
]

```

加载转换验证后的embedding_model输出的相似度:

二、数据集转换

1. 选择模型微调工具，根据微调框架选择对应数据集格式（此处以xtuner为例，使用xtuner默认格式）

- 数据集转换代码

```
import json

def convert_format(source_data):
    target_data = []
    for item in source_data:
        # 构建新的对话格式
        new_convo = {
            "conversation": [
                {
                    "input": item["user"],
                    "output": f"{item['style']}\n{item['assistant']}"
                }
            ]
        }
        target_data.append(new_convo)
    return target_data

# 从文件读取源数据
with open("input.json", "r", encoding="utf-8") as f:
    source_data = json.load(f)

# 执行转换
converted_data = convert_format(source_data)

# 写入目标文件
with open("output.json", "w", encoding="utf-8") as f:
    json.dump(converted_data, f, ensure_ascii=False, indent=2)
```

三、模型选型

1. 根据任务选择对应的评测数据，对预期模型客观评测

当前任务为日常聊天对话模型，主要要求模型的中文理解能力，因此这里以CLUE（中文理解）数据进行评测：

```
#输出数据集清单
python tools/list_configs.py clue

#输出如下：
+-----+-----+
| Dataset | Config Path |
|-----+-----|
|         |             |
+-----+-----+
```

CLUE_C3_gen	
opencompass/configs/datasets/CLUE_C3/CLUE_C3_gen.py	
CLUE_C3_gen_8c358f	
opencompass/configs/datasets/CLUE_C3/CLUE_C3_gen_8c358f.py	
CLUE_C3_pp1	
opencompass/configs/datasets/CLUE_C3/CLUE_C3_pp1.py	
CLUE_C3_pp1_56b537	
opencompass/configs/datasets/CLUE_C3/CLUE_C3_pp1_56b537.py	
CLUE_C3_pp1_e24a31	
opencompass/configs/datasets/CLUE_C3/CLUE_C3_pp1_e24a31.py	
CLUE_CMRC_gen	
opencompass/configs/datasets/CLUE_CMRC/CLUE_CMRC_gen.py	
CLUE_CMRC_gen_1bd3c8	
opencompass/configs/datasets/CLUE_CMRC/CLUE_CMRC_gen_1bd3c8.py	
CLUE_CMRC_gen_3749cd	
opencompass/configs/datasets/CLUE_CMRC/CLUE_CMRC_gen_3749cd.py	
CLUE_CMRC_gen_8484b9	
opencompass/configs/datasets/CLUE_CMRC/CLUE_CMRC_gen_8484b9.py	
CLUE_CMRC_gen_941108	
opencompass/configs/datasets/CLUE_CMRC/CLUE_CMRC_gen_941108.py	
CLUE_DRCD_gen	
opencompass/configs/datasets/CLUE_DRCD/CLUE_DRCD_gen.py	
CLUE_DRCD_gen_1bd3c8	
opencompass/configs/datasets/CLUE_DRCD/CLUE_DRCD_gen_1bd3c8.py	
CLUE_DRCD_gen_3749cd	
opencompass/configs/datasets/CLUE_DRCD/CLUE_DRCD_gen_3749cd.py	
CLUE_DRCD_gen_8484b9	
opencompass/configs/datasets/CLUE_DRCD/CLUE_DRCD_gen_8484b9.py	
CLUE_DRCD_gen_941108	
opencompass/configs/datasets/CLUE_DRCD/CLUE_DRCD_gen_941108.py	
CLUE_afqmc_gen	
opencompass/configs/datasets/CLUE_afqmc/CLUE_afqmc_gen.py	
CLUE_afqmc_gen_901306	
opencompass/configs/datasets/CLUE_afqmc/CLUE_afqmc_gen_901306.py	
CLUE_afqmc_pp1	
opencompass/configs/datasets/CLUE_afqmc/CLUE_afqmc_pp1.py	
CLUE_afqmc_pp1_378c5b	
opencompass/configs/datasets/CLUE_afqmc/CLUE_afqmc_pp1_378c5b.py	
CLUE_afqmc_pp1_6507d7	
opencompass/configs/datasets/CLUE_afqmc/CLUE_afqmc_pp1_6507d7.py	
CLUE_afqmc_pp1_7b0c1e	
opencompass/configs/datasets/CLUE_afqmc/CLUE_afqmc_pp1_7b0c1e.py	
CLUE_cmnli_gen	
opencompass/configs/datasets/CLUE_cmnli/CLUE_cmnli_gen.py	
CLUE_cmnli_gen_1abf97	
opencompass/configs/datasets/CLUE_cmnli/CLUE_cmnli_gen_1abf97.py	
CLUE_cmnli_gen_51e956	
opencompass/configs/datasets/CLUE_cmnli/CLUE_cmnli_gen_51e956.py	
CLUE_cmnli_pp1	
opencompass/configs/datasets/CLUE_cmnli/CLUE_cmnli_pp1.py	
CLUE_cmnli_pp1_98dd6e	
opencompass/configs/datasets/CLUE_cmnli/CLUE_cmnli_pp1_98dd6e.py	
CLUE_cmnli_pp1_ef69e7	
opencompass/configs/datasets/CLUE_cmnli/CLUE_cmnli_pp1_ef69e7.py	
CLUE_cmnli_pp1_fdc6de	
opencompass/configs/datasets/CLUE_cmnli/CLUE_cmnli_pp1_fdc6de.py	
CLUE_ocnli_gen	
opencompass/configs/datasets/CLUE_ocnli/CLUE_ocnli_gen.py	

CLUE_ocnli_gen_51e956	
opencompass/configs/datasets/CLUE_ocnli/CLUE_ocnli_gen_51e956.py	
CLUE_ocnli_gen_c4cb6c	
opencompass/configs/datasets/CLUE_ocnli/CLUE_ocnli_gen_c4cb6c.py	
CLUE_ocnli_pp1	
opencompass/configs/datasets/CLUE_ocnli/CLUE_ocnli_pp1.py	
CLUE_ocnli_pp1_98dd6e	
opencompass/configs/datasets/CLUE_ocnli/CLUE_ocnli_pp1_98dd6e.py	
CLUE_ocnli_pp1_ef69e7	
opencompass/configs/datasets/CLUE_ocnli/CLUE_ocnli_pp1_ef69e7.py	
CLUE_ocnli_pp1_fdc6de	
opencompass/configs/datasets/CLUE_ocnli/CLUE_ocnli_pp1_fdc6de.py	
FewCLUE_bustm_gen	
opencompass/configs/datasets/FewCLUE_bustm/FewCLUE_bustm_gen.py	
FewCLUE_bustm_gen_634f41	
opencompass/configs/datasets/FewCLUE_bustm/FewCLUE_bustm_gen_634f41.py	
FewCLUE_bustm_pp1	
opencompass/configs/datasets/FewCLUE_bustm/FewCLUE_bustm_pp1.py	
FewCLUE_bustm_pp1_4b16c0	
opencompass/configs/datasets/FewCLUE_bustm/FewCLUE_bustm_pp1_4b16c0.py	
FewCLUE_bustm_pp1_9ef540	
opencompass/configs/datasets/FewCLUE_bustm/FewCLUE_bustm_pp1_9ef540.py	
FewCLUE_bustm_pp1_e53034	
opencompass/configs/datasets/FewCLUE_bustm/FewCLUE_bustm_pp1_e53034.py	
FewCLUE_chid_gen	
opencompass/configs/datasets/FewCLUE_chid/FewCLUE_chid_gen.py	
FewCLUE_chid_gen_0a29a2	
opencompass/configs/datasets/FewCLUE_chid/FewCLUE_chid_gen_0a29a2.py	
FewCLUE_chid_pp1	
opencompass/configs/datasets/FewCLUE_chid/FewCLUE_chid_pp1.py	
FewCLUE_chid_pp1_8f2872	
opencompass/configs/datasets/FewCLUE_chid/FewCLUE_chid_pp1_8f2872.py	
FewCLUE_chid_pp1_accb5	
opencompass/configs/datasets/FewCLUE_chid/FewCLUE_chid_pp1_accb5.py	
FewCLUE_cluewsc_gen	
opencompass/configs/datasets/FewCLUE_cluewsc/FewCLUE_cluewsc_gen.py	
FewCLUE_cluewsc_gen_c68933	
opencompass/configs/datasets/FewCLUE_cluewsc/FewCLUE_cluewsc_gen_c68933.py	
FewCLUE_cluewsc_pp1	
opencompass/configs/datasets/FewCLUE_cluewsc/FewCLUE_cluewsc_pp1.py	
FewCLUE_cluewsc_pp1_12e4e0	
opencompass/configs/datasets/FewCLUE_cluewsc/FewCLUE_cluewsc_pp1_12e4e0.py	
FewCLUE_cluewsc_pp1_4284a0	
opencompass/configs/datasets/FewCLUE_cluewsc/FewCLUE_cluewsc_pp1_4284a0.py	
FewCLUE_cluewsc_pp1_868415	
opencompass/configs/datasets/FewCLUE_cluewsc/FewCLUE_cluewsc_pp1_868415.py	
FewCLUE_csl_gen	
opencompass/configs/datasets/FewCLUE_csl/FewCLUE_csl_gen.py	
FewCLUE_csl_gen_28b223	
opencompass/configs/datasets/FewCLUE_csl/FewCLUE_csl_gen_28b223.py	
FewCLUE_csl_gen_87f4a8	
opencompass/configs/datasets/FewCLUE_csl/FewCLUE_csl_gen_87f4a8.py	
FewCLUE_csl_pp1	
opencompass/configs/datasets/FewCLUE_csl/FewCLUE_csl_pp1.py	
FewCLUE_csl_pp1_769f8d	
opencompass/configs/datasets/FewCLUE_csl/FewCLUE_csl_pp1_769f8d.py	
FewCLUE_csl_pp1_841b62	
opencompass/configs/datasets/FewCLUE_csl/FewCLUE_csl_pp1_841b62.py	

FewCLUE_eprstmt_gen	
opencompass/configs/datasets/FewCLUE_eprstmt/FewCLUE_eprstmt_gen.py	
FewCLUE_eprstmt_gen_740ea0	
opencompass/configs/datasets/FewCLUE_eprstmt/FewCLUE_eprstmt_gen_740ea0.py	
FewCLUE_eprstmt_pp1	
opencompass/configs/datasets/FewCLUE_eprstmt/FewCLUE_eprstmt_pp1.py	
FewCLUE_eprstmt_pp1_1ce587	
opencompass/configs/datasets/FewCLUE_eprstmt/FewCLUE_eprstmt_pp1_1ce587.py	
FewCLUE_eprstmt_pp1_f1e631	
opencompass/configs/datasets/FewCLUE_eprstmt/FewCLUE_eprstmt_pp1_f1e631.py	
FewCLUE_ocnli_fc_gen	
opencompass/configs/datasets/FewCLUE_ocnli_fc/FewCLUE_ocnli_fc_gen.py	
FewCLUE_ocnli_fc_gen_f97a97	
opencompass/configs/datasets/FewCLUE_ocnli_fc/FewCLUE_ocnli_fc_gen_f97a97.py	
FewCLUE_ocnli_fc_pp1	
opencompass/configs/datasets/FewCLUE_ocnli_fc/FewCLUE_ocnli_fc_pp1.py	
FewCLUE_ocnli_fc_pp1_9e8b3d	
opencompass/configs/datasets/FewCLUE_ocnli_fc/FewCLUE_ocnli_fc_pp1_9e8b3d.py	
FewCLUE_ocnli_fc_pp1_c08300	
opencompass/configs/datasets/FewCLUE_ocnli_fc/FewCLUE_ocnli_fc_pp1_c08300.py	
FewCLUE_tnews_gen	
opencompass/configs/datasets/FewCLUE_tnews/FewCLUE_tnews_gen.py	
FewCLUE_tnews_gen_b90e4a	
opencompass/configs/datasets/FewCLUE_tnews/FewCLUE_tnews_gen_b90e4a.py	
FewCLUE_tnews_pp1	
opencompass/configs/datasets/FewCLUE_tnews/FewCLUE_tnews_pp1.py	
FewCLUE_tnews_pp1_7d1c07	
opencompass/configs/datasets/FewCLUE_tnews/FewCLUE_tnews_pp1_7d1c07.py	
FewCLUE_tnews_pp1_d10e8a	
opencompass/configs/datasets/FewCLUE_tnews/FewCLUE_tnews_pp1_d10e8a.py	
FewCLUE_tnews_pp1_fff486	
opencompass/configs/datasets/FewCLUE_tnews/FewCLUE_tnews_pp1_fff486.py	
+-----+	
-----+	

当前任务大多是短语对话，可以选择 FewCLUE_bustm_gen（短文本分类）、FewCLUE_ocnli_fc_gen（自然语言推理）对预期模型进行评估。

```
python run.py \
  --models hf_qwen1_5_0_5b_chat hf_qwen1_5_1_8b_chat \
  --datasets FewCLUE_bustm_gen FewCLUE_ocnli_fc_gen \
  --debug
```

根据评估结果，选择最终模型。

四、模型训练

1. 配置训练文件

PART 1中

#预训练模型存放的位置

pretrained_model_name_or_path = 'model_path'#基座模型路径

#微调数据存放的位置

```
data_files = '/root/public/data/target_data.json'
```

训练中最大的文本长度

```
max_length = 512
```

每一批训练样本的大小

```
batch_size = 2
```

#最大训练轮数

```
max_epochs = 3
```

#验证数据

```
evaluation_inputs = [  
    '只剩一个心脏了还能活吗?', '爸爸再婚，我是不是就有了个新娘?',  
    '樟脑丸是我吃过最难吃的硬糖有奇怪的味道怎么还有人买', '马上要上游泳课了，昨天洗的泳裤还没干，怎么办',  
    '我只出生了一次，为什么每年都要庆生'  
]
```

PART 3中

```
dataset=dict(type=load_dataset, path="json", data_files=data_files)  
dataset_map_fn=None
```

2. 在当前目录下，输入以下命令启动微调脚本

#单机单卡

```
xtuner train internlm2_chat_1_8b_qlora_alpaca_e3.py
```

#单机多卡

```
NPROC_PER_NODE=${GPU_NUM} xtuner train internlm2_chat_7b_qlora_oasst1_e3 --  
deepspeed deepspeed_zero2
```

#后台执行

```
nohup bash -c 'NPROC_PER_NODE=2 xtuner train  
qwen1_5_1_8b_chat_qlora_alpaca_e3.py --deepspeed deepspeed_zero2' > train.log  
2>&1 &
```

3.模型转换

模型训练后会自动保存成 PTH 模型（例如 `iter_2000.pth`，如果使用了 DeepSpeed，则将会是一个文件夹），我们需要利用 `xtuner convert pth_to_hf` 将其转换为 HuggingFace 模型，以便于后续使用。具体命令为：

```
xtuner convert pth_to_hf ${FINETUNE_CFG} ${PTH_PATH} ${SAVE_PATH}  
# 例如: xtuner convert pth_to_hf internlm2_chat_7b_qlora_custom_sft_e1_copy.py  
./iter_2000.pth ./iter_2000_
```


4.模型合并

如果使用了 LoRA / QLoRA 微调，则模型转换后将得到 adapter 参数，而并不包含原 LLM 参数。如果您期望获得合并后的模型权重（例如用于后续评测），那么可以利用 `xtuner convert merge`：

```
$ xtuner convert merge ${LLM} ${LLM_ADAPTER} ${SAVE_PATH}
```

五、模型推理部署

选择合适的大模型推理框架部署模型（这里选择LMDeploy），须注意对话模板对其操作！

LMDeploy 支持两种添加对话模板的形式：

- 一种是利用现有对话模板，直接配置一个如下的 json 文件使用。

```
{
  "model_name": "your awesome chat template name",
  "system": "<|im_start|>system\n",
  "meta_instruction": "You are a robot developed by LMDeploy.",
  "eosys": "<|im_end|>\n",
  "user": "<|im_start|>user\n",
  "eoh": "<|im_end|>\n",
  "assistant": "<|im_start|>assistant\n",
  "eoa": "<|im_end|>",
  "separator": "\n",
  "capability": "chat",
  "stop_words": ["<|im_end|>"]
}
```

model_name 为必填项，可以是 LMDeploy 内置对话模板名（通过 `lmdeploy list` 可查阅），也可以是新名字。其他字段可选填。当 model_name 是内置对话模板名时，json 文件中各非 null 字段会覆盖原有对话模板的对应属性。而当 model_name 是新名字时，它会吧将 `BaseChatTemplate` 直接注册成新的对话模板。其具体定义可以参考 [BaseChatTemplate](#)。这样一个模板将会以下面的形式进行拼接。

```
{system}{meta_instruction}{eosys}{user}{user_content}{eoh}{assistant}
{assistant_content}{eoa}{separator}{user}...
```

在使用 CLI 工具时，可以通过 `--chat-template` 传入自定义对话模板，比如：

```
lmdeploy serve api_server internlm/internlm2_5-7b-chat --chat-template
${JSON_FILE}
```

也可以在通过接口函数传入，比如：


```
from lmdeploy import ChatTemplateConfig, serve

serve('internlm/internlm2_5-7b-chat',
      chat_template_config=ChatTemplateConfig.from_json('${JSON_FILE}'))
```

- 一种是以 LMDeploy 现有对话模板，自定义一个python对话模板类，注册成功后直接用即可。优点是自定义程度高，可控性强。下面是一个注册 LMDeploy 对话模板的例子：

```
from lmdeploy.model import MODELS, BaseChatTemplate

@MODELS.register_module(name='customized_model')
class CustomizedModel(BaseChatTemplate):
    """A customized chat template."""

    def __init__(self,
                 system='<|im_start|>system\n',
                 meta_instruction='You are a robot developed by LMDeploy.',
                 user='<|im_start|>user\n',
                 assistant='<|im_start|>assistant\n',
                 eosys='<|im_end|>\n',
                 eoh='<|im_end|>\n',
                 eoa='<|im_end|>',
                 separator='\n',
                 stop_words=['<|im_end|>', '<|action_end|>']):
        super().__init__(system=system,
                         meta_instruction=meta_instruction,
                         eosys=eosys,
                         user=user,
                         eoh=eoh,
                         assistant=assistant,
                         eoa=eoa,
                         separator=separator,
                         stop_words=stop_words)

from lmdeploy import ChatTemplateConfig, pipeline

messages = [{'role': 'user', 'content': 'who are you?'}]
pipe = pipeline('internlm/internlm2_5-7b-chat',
                chat_template_config=ChatTemplateConfig('customized_model'))
for response in pipe.stream_infer(messages):
    print(response.text, end='')
```

这里我们选用CLI 工具推理，可以通过 `--chat-template` 传入自定义对话模板：

```
lmdeploy serve api_server internlm/internlm2_5-7b-chat --chat-template
${JSON_FILE}
```

对话模板转换脚本：

```
import re
import json
```

```

from typing import Dict, Any

def universal_converter(original_template: Dict[str, Any]) -> Dict[str, Any]:
    """将多种风格的原始模板转换为lmdeploy官方格式"""

    # 字段映射关系（核心逻辑）
    field_mapping = {
        # 基础字段映射
        "SYSTEM": "system",
        "INSTRUCTION": ("user", "assistant"), # 需要拆分处理
        "SUFFIX": "eoa",
        "SEP": "separator",
        "STOP_WORDS": "stop_words",

        # 特殊处理字段
        "SUFFIX_AS_EOS": None, # 该字段在官方模板中不需要
    }

    # 初始化目标模板（包含必填字段默认值）
    converted = {
        "meta_instruction": "You are a helpful assistant.", # 必填项
        "capability": "chat", # 必填项
        "eosys": "<|im_end|>\n", # 通常固定格式
        "eoh": "<|im_end|>\n", # 通常固定格式
    }

    # 自动处理字段映射
    for src_key, dest_key in field_mapping.items():
        if src_key in original_template:
            value = original_template[src_key]

            # 处理需要拆分的字段（如INSTRUCTION）
            if isinstance(dest_key, tuple) and src_key == "INSTRUCTION":
                # 使用正则拆分user和assistant部分
                parts = re.split(r'(<|im_start|>assistant\n?)', value)
                converted["user"] = parts[0].strip()
                if len(parts) > 1:
                    converted["assistant"] = parts[1] + parts[2] if len(parts) >
2 else parts[1]

            # 处理直接映射字段
            elif dest_key and not isinstance(dest_key, tuple):
                converted[dest_key] = value

    # 特殊处理system字段的占位符
    if "system" in converted:
        converted["system"] = converted["system"].replace("{system}", "{{ system
}}")

    # 处理用户输入占位符
    if "user" in converted:
        converted["user"] = converted["user"].replace("{input}", "{{ input }}")

    # 自动处理停止词（兼容列表和字符串）
    if "stop_words" in converted and isinstance(converted["stop_words"], str):
        converted["stop_words"] = [converted["stop_words"]]

```

```

# 保留原始模板中的额外字段（带警告）
for key in original_template:
    if key not in field_mapping:
        print(f"warning: 发现未映射字段 [{key}], 已保留原样")
        converted[key] = original_template[key]

return converted

# 示例用法
original_qwen_chat = dict(
    SYSTEM=("<|im_start|>system\n{system}<|im_end|>\n"),
    INSTRUCTION=("<|im_start|>user\n{input}<|im_end|>\n" "
<|im_start|>assistant\n"),
    SUFFIX="<|im_end|>",
    SUFFIX_AS_EOS=True,
    SEP="\n",
    STOP_WORDS=["<|im_end|>", "<|endoftext|>"]
)

# 执行转换
converted_template = universal_converter(original_qwen_chat)

# 生成JSON文件
with open('chat_template.json', 'w') as f:
    json.dump(converted_template, f,
        indent=2,
        ensure_ascii=False,
        separators=(',', ':'))

```

六、模型推理部署

使用下面命令启动streamlit web前端，测试最终效果：

```
streamlit run chat_app.py
```

最终效果如下：

项目一效果演示



同事老是在背后背刺我。



溫柔 哎呀，这听起来真的很让人不舒服呢。❤️ 想要谈谈具体情况吗？我们可以一起想想办法解决问题。🌸



拍Vlog被猴抢包，素材变动物世界！



毒舌 桂Q，你这拍的不是Vlog，这是《人与自然》的现场版吧！下次记得带上猴群认证的摄影师，保佑素材别变成现实版的“人与自然”。😂安全第一，别让小孩子影响了你的高颜值Vlog。📷



老妈非让我嫁给她同事儿子，怎么逃啊！



溫柔 哎呀，这种情况下逃婚确实是个难题呢。不过，婚姻大事，最重要的是你自己的意愿哦。❤️

你可以跟妈妈沟通一下，表达一下你的想法和顾虑，同时也要考虑清楚嫁给他同事儿子的影响会有多大。如果需要帮助想出解决办法，或者只是想要倾诉一下，我都在这里哦。🌸



老妈非让我嫁给她同事儿子，怎么逃啊！

请输入您的问题，或输入exit退出

